第10章 SQLite数据库

数据！ 数据！ 数据！

目　录

10.1 sqlite数据库概述
10.2 sqlite安装
10.3 sqlite的常用命令
10.4 sqlite的数据类型
10.5 sqlite的API函数

嵌入式数据库系统是指支持移动计算或某种特定计算模式的数据库管理系统，它通常与操作系统和具体应用集成在一起，运行在智能型嵌入式设备或移动设备上。

由于嵌入式数据库系统总是与移动计算相结合，所以通常情况下嵌入式数据库也被称为嵌入式移动数据库。

嵌入式数据库系统具备了如下主要特点：

（1）嵌入性

（2） 移植性

（3） 安全性

（4）实时性

（5）可靠性

（6）主动性

嵌入式数据库的分类方法很多，根据其嵌入的对象不同分为：
面向软件的嵌入式数据库、
面向设备的嵌入式数据库、
内存数据库等。

也可以根据其应用的不同分为：
普通嵌入式数据库、
嵌入式移动数据库、
小型架构数据库等。

在众多的数据库中，如何选择适用于嵌入式系统的数据库呢？嵌入式系统开发环境决定了其对数据库需求的要素：

（1） 体积较小

（2） 功能齐备

（3） 代码开源

（4） 性能可靠

10.1

Part One

sqlite数据库概述

10.1.1基于Linux平台的嵌入式数据库概述

基于Linux平台的数据库非常多，大型的商用数据库有Oracle、Sybase、Informix、Informix、IBM DB2等;

中小型的更是不胜枚举，最常见的几种是：PostgreSQL、MySQL、mSQL、Berkeley DB和SQLite数据库。

MySQL是多用户、多进程的SQL数据库系统

- MySQL既能够作为一个单独的应用程序应用在客户端服务器网络环境中，也能够作为一个库而嵌入到其他的软件中。
- 使用 C和 C++编写，并使用了多种编译器进行测试，保证了源代码的可移植性。
- MySQL为多种编程语言提供了 API。支持多线程，充分利用 CPU 资源。
- MySQL 软件采用了双授权政策，它分为社区版和商业版
- 由于其体积小、速度快、总体拥有成本低，尤其是开放源码这一特点，一般中小型网站的开发常选择 MySQL 作为网站数据库。

开源数据库性能对比表

产品名称	速度	稳定性	数据库容量	SQL支持	Win32平台下最小体积	数据操纵

王老师，咱们组想的模型有一个是CNN人脸检测+Resnet人脸识别+Sqlite3数据库+FPGA拨号模块；还有一个是mobilenet +facenet +SQLite 数据库+FPGA拨号模块，请您看看哪个合适吖😁

那这两个各有哪个优势呢?

第一个准确度高一些，第二个是轻量化的检测模型

10.1.2 SQLite的特点

SQLite是一个开源的、内嵌式的关系型数据库。它是D. Richard Hipp采用C语言开发出来的完全独立的，不具有外部依赖性的嵌入式数据库引擎。SQLite第一个版本发布于2000年5月，在便携性、易用性、紧凑性、有效性和可靠性方面有突出的表现。

四个特性：原子性（atomicity，或称不可分割性）、一致性（consistency）、隔离性（isolation，又称独立性）、持久性（durability）。

Sqlite主要特点有：

支持ACID事务。

零配置，即无需安装和管理配置。

储存在单一磁盘文件中的一个完整的数据库。

数据文件可在不同字节顺序的机器间自由共享。

支持数据库大小至2TB。

程序体积小，全部C语言代码约3万行（核心软件，包括库和工具)，250KB大小。

相对于目前其他嵌入式数据库具有更快捷的数据操作。

支持事务功能和并发处理。

程序完全独立，不具有外部依赖性。

支持多种硬件平台，如arm/ Linux、SPARC/Solaris等。

10.1.3 sqlite的体系结构

Sqlite主要由core（核心），backend（后端）和accessories（附件）三个子系统组成。

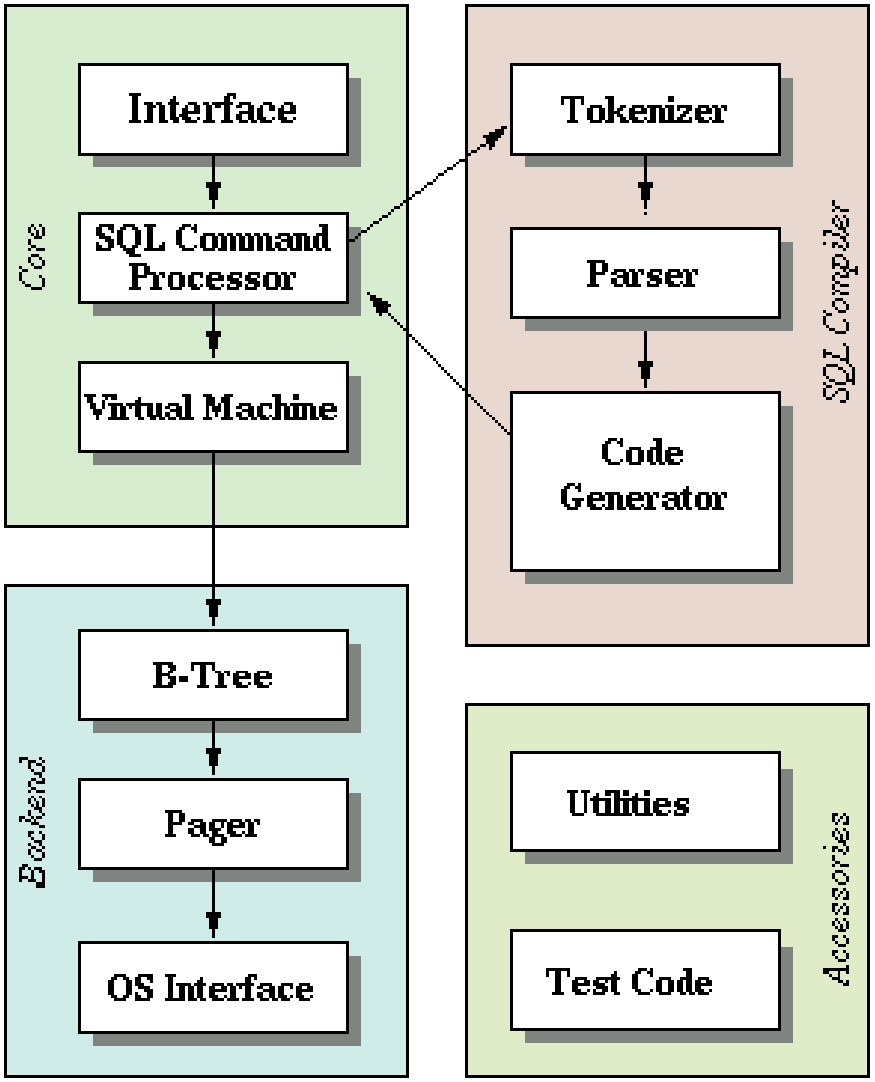

图10-1 sqlite的体系结构

10.2

Part Two

sqlite安装

SQLite网站(www.sqlite.org)同时提供SQLite的已编译版本和源程序。编译版本可同时适用于Windows和Linux。
有几种形式的二进制包供选择，以适应SQLite的不同使用方式。包括：静态链接的命令行程序(CLP)、SQLite动态链接库(DLL)和Tcl扩展。

SQLite源代码以两种形式提供，以适应不同的平台：
一种为了在Windows下编译，
另一种为了在POSIX平台(如Linux、BSD、Solaris)下编译

10.3 Part Three

sqlite的常用命令

启动sqlite3程序，仅仅需要敲入带有SQLite数据库名字的"sqlite3"命令即可。如果文件不存在，则创建一个新的数据库文件。然后 sqlite3程序将提示输入SQL，输入SQL语句（以分号“；”结束）并回车之后，SQL语句就会执行。

例如，创建一个名字为 “test”的SQLite数据库，如下：

```
sqlite3 test
SQLite version 3.7.14
Enter ".help" for instructions
sqlite>
```

10.4 Part FouR

sqlite的数据类型

SQLite采用的是弱类型的数据。SQLite 3.0及以后版本支持更多的数据类型。每个数据值本身的数据类型可以是下列五种类型对象之一：

（1）NULL，空值；

（2）INTEGER，整型，根据大小使用1、2、3、4、6、8个字节来存储；

（3）REAL，浮点型，用来存储8个字节的IEEE浮点；

（4）TEXT，文本字符串，使用UTF-8、UTF-16、UTF-32等保存数据；

（5）BLOB（Binary Large Objects），二进制类型，按照二进制存储，不做任何改变。

表10-3 sqlite3接受的数据类型（扩展）

数据类型	说明
smallint	16 位元的整数
interger	32 位元的整数
decimal(p，s)	精确值p是指全部有几个数(digits)大小值，s是指小数点後有几位数。如未特别指定，则默认设为p=5; s=0
float	32位元的实数
double	64位元的实数
char(n)	n 长度的字串，n不能超过 254
varchar(n)	长度不固定且其最大长度为 n 的字串，n不能超过4000
graphic(n)	类似char(n)，但其单位是两个字元 double-bytes，n不能超过127。
vargraphic(n)	可变长度且其最大长度为 n 的双字元字串，n不能超过 2000

为了增强SQLite数据库和其他数据库列类型的兼容性，SQLite支持列的“类型亲和性”。列的亲和性是为该列所存储的数据建议一个类型（注意是建议而不是指定）。对于某些列，如果给出了建议类型，数据库将按建议的类型先转换再存储，这个被优先使用的数据类型则被称为"亲和类型"。

在SQLite3.0版中数据库中的列类型有五种类型亲和性：文本类型、数字类型、整数类型、浮点类型、NULL无类型。

10.5

Part FIVE

sqlite的API函数

从功能的角度来区分，SQLite的API可分为两类：核心API和扩充API。

核心API由所有完成基本数据库操作的函数构成，主要包括连接数据库、执行SQL和遍历结果集等。它还包括一些功能函数，用来完成字符串格式化、操作控制、调试和错误处理等任务。

扩充API提供不同的方法来扩展SQLite，它向用户提供创建自定义的SQL扩展，并与SQLite本身的SQL相集成等功能。

10.5.1核心C API函数

核心C API主要与执行SQL命令有关。核心C API大约有十个。它们分别是：

sqlite3_open()
sqlite3_prepare()
sqlite3_step()
sqlite3_column()
sqlite3_finalize()
sqlite3_close()
sqlite3_exec()
sqlite3_get_table()
sqlite3_reset()
sqlite3_bind()

有两种方法执行SQL语句：预编译查询(Prepared Query)和封装查询。

预编译查询由三个阶段构成：准备(preparation)、执行(execution)和定案(finalization)。

而封装查询只是对预编译查询的三个过程进行了封装，最终也会转化为预编译查询来执行。

预处理查询是SQLite执行所有SQL命令的方式，主要包括以下三个步聚：

(1) 准备

分词器、分析器和代码生成器把SQL语句编译成虚拟机字节码，编译器会创建一个语句句柄(sqlite3_stmt)，它包括字节码以及其它执行命令和遍历结果集所需的全部资源。相应的C API为sqlite3_prepare()，位于prepare.c文件中，有多种类似的形式，如sqlite3_prepare()、sqlite3_prepare16()、sqlite3_prepare_v2()等。

完整的API语法是：

```
int sqlite3_prepare(
  sqlite3 *db,            /* db为sqlite3的句柄*/
  const char *zSql,       /* zSql为要执行的SQL语句 */
  int nByte,              /* nByte为要执行语句在zSql中的最大长度，如果是负数，那么就需要重新自动计算 */
  sqlite3_stmt **ppStmt,  /* ppStmt为预编译后的句柄 */
  const char **pzTail     /* pzTail预编译后剩下的字符串（未预编译成功或者多余的）的指针，一般传入0或者NULL即可*/
```

(2) 执行

虚拟机执行字节码，执行过程是一个步进(stepwise)的过程，每一步由sqlite3_step()启动，并由虚拟机执行一段字节码。

(3) 定案

虚拟机关闭语句，释放资源。相应的C API为sqlite3_finalize()，它导致虚拟机结束程序运行并关闭语句句柄。如果事务是由人工控制开始的，它必须由人工控制进行提交或回卷，否则sqlite3_finalize()会返回一个错误。当sqlite3_finalize()执行成功，所有与语句对象关联的资源都将被释放。在自动提交模式下，还会释放关联的数据库锁。

2.sqlite3_open()或sqlite3_open16()函数

打开数据库用sqlite3_open()或sqlite3_open16()函数，它们的声明如下：

```
int sqlite3_open(
    const char *filename，                    /* 数据库文件名 (UTF-8) */
    sqlite3 **ppDb                            /* 输出数据库句柄 */
);
int sqlite3_open16(
    const void *filename，                    /* 数据库文件名 (UTF-16) */
    sqlite3 **ppDb                            /* 输出数据库句柄 */
);
```

3.sqlite3_close()函数

关闭数据库用sqlite3_close()函数，声明如下：

int sqlite3_close(sqlite3*);

为了sqlite3_close()能够成功执行，所有与连接所关联的且已编译的查询必须被定案。如果仍然有查询没有定案，sqlite3_close()将返回SQLITE_BUSY和错误信息。

4.sqlite3_exec（）函数

大部分sql操作都可以通过sqlite3_exec来完成，它的API形式如下：

```
int sqlite3_exec(
    sqlite3*,                                   /* 数据库句柄 */
    const char *sql,                            /* 要执行的SQL语句*/
    int (*callback)(void*,int,char**,char**),  /* callback回调函数 */
    void *,                                     /* void *回调函数的第一个参数 */
    char **errmsg                               /* errmsg错误信息，如果没有SQL问题则值为NULL */
    );
```

回调函数是一个比较复杂的函数。原型如下：

```
int callback(void *params,int column_size,char **column_value,char **column_name)
```

参数说明如下：

params是sqlite3_exec传入的第四个参数；

column_size是结果字段的个数；

column_value是返回记录的一位字符数组指针；

column_name是结果字段的名称；

4. sqlite3_bind

SQL声明可以包含一些型如“?” 或 “?nnn” 或 “:aaa”的标记， 其中“nnn” 是一个整数，“aaa” 是一个字符串. 这些标记代表一些不确定的字符值（或者通配符），用户可以在后面用sqlite3_bind 接口来填充这些值。

5. sqlite3_get_table（）函数

sqlite3_get_table的说明如下：

```
int sqlite3_get_table(
  sqlite3 *db,              /* db是sqlite3的句柄*/
  const char *zSql,       /* zSql是要执行的sql语句 */
  char ***pazResult,     /* pazResult是执行查询操作的返回结果集 */
  int *pnRow,              /* pnRow是记录的行数*/
  int *pnColumn,          /* pnColumn是记录的字段个数 */
  char **pzErrmsg        /* pzErrmsg是错误信息 */
);
```

由于sqlite3_get_table是sqlite3_exec的包装，因此返回的结果和sqlite3_exec类似。

10.5.2 扩充C API 函数

SQLite的扩充API用来支持用户定义的函数、聚合和排序法。用户定义函数是一个SQL函数，它对应于用C语言或其它语言实现的函数的句柄。使用C API时，这些句柄用C/C++实现。用户定义的扩展必须注册到一个由连接到连接的基础之上，存储在程序内存中。也就是说，它们不是存储在数据库中，而是存储在用户的程序中。